DATA PREPROCESSING AND FEATURE ENGINEERING FOR CREDIT CARD FRAUD DETECTION

Data Mining Assignment 1

Vinayak Vemula

Dataset Description

Source: TensorFlow / Kaggle (public dataset)
URL: https://storage.googleapis.com/download.tensorflow.org/data/creditcard.csv
Description: Contains credit card transactions made by European cardholders in September 2013.

Key Stats:

- **Shape:** 284,807 rows × 31 columns
- **Features:**

  Time: *Seconds since first transaction*

  V1–V28: *PCA-transformed components (anonymize*

  Amount: *Transaction amount*

  Class: *Target variable (1 = Fraud, 0 = Non-Fraud)*

```
Dataset loaded successfully.
Shape of dataset: (284807, 31)
   Time        V1        V2        V3        V4        V5        V6        V7  \
0   0.0 -1.359807 -0.072781  2.536347  1.378155 -0.338321  0.462388  0.239599
1   0.0  1.191857  0.266151  0.166480  0.448154  0.060018 -0.082361 -0.078803
2   1.0 -1.358354 -1.340163  1.773209  0.379780 -0.503198  1.800499  0.791461
3   1.0 -0.966272 -0.185226  1.792993 -0.863291 -0.010309  1.247203  0.237609
4   2.0 -1.158233  0.877737  1.548718  0.403034 -0.407193  0.095921  0.592941

          V8        V9  ...       V21       V22       V23       V24       V25  \
0  0.098698  0.363787  ... -0.018307  0.277838 -0.110474  0.066928  0.128539
1  0.085102 -0.255425  ... -0.225775 -0.638672  0.101288 -0.339846  0.167170
2  0.247676 -1.514654  ...  0.247998  0.771679  0.909412 -0.689281 -0.327642
3  0.377436 -1.387024  ... -0.108300  0.005274 -0.190321 -1.175575  0.647376
4 -0.270533  0.817739  ... -0.009431  0.798278 -0.137458  0.141267 -0.206010

        V26       V27       V28  Amount  Class
0 -0.189115  0.133558 -0.021053  149.62      0
1  0.125895 -0.008983  0.014724    2.69      0
2 -0.139097 -0.055353 -0.059752  378.66      0
3 -0.221929  0.062723  0.061458  123.50      0
4  0.502292  0.219422  0.215153   69.99      0

[5 rows x 31 columns]
```

Preprocessing: Data Cleaning & Transformation

Outlier Removal (Z-score)

- **Operation:** Removed transactions where the ‘Amount’ Z-score > 3.
- **Justification:** Eliminates extreme values that could bias the model, improving stability and accuracy.

```python
# 2. Data Cleaning
print("Checking for Missing Values")
missing_values = df.isnull().sum()
print(missing_values[missing_values > 0])

# Handle missing values
df = df.dropna()

# Outlier Detection
# Detect outliers in the 'Amount' feature using Z-score
df['Zscore_Amount'] = (df['Amount'] - df['Amount'].mean()) / df['Amount'].std()
df['Outlier'] = df['Zscore_Amount'].apply(lambda x: 1 if abs(x) > 3 else 0)
print(f"\nNumber of detected outliers: {df['Outlier'].sum()}")

# Optionally remove outliers
df = df[df['Outlier'] == 0].drop(columns=['Zscore_Amount', 'Outlier'])
```

```
Checking for Missing Values
Series([], dtype: int64)

Number of detected outliers: 6711
```

Feature Scaling (StandardScaler):

- **Operation:** Scaled ‘Amount’ and ‘Time’ using Z-score normalization.
- **Justification:** Ensures all features (including PCA-transformed V1–V28) are on comparable scales for better model performance.

```python
# 3. Data Transformation
print("Feature Scaling")
scaler = StandardScaler()
df['Amount_Scaled'] = scaler.fit_transform(df[['Amount']])
df['Time_Scaled'] = scaler.fit_transform(df[['Time']])

# Drop the original unscaled columns
df_scaled = df.drop(columns=['Amount', 'Time'])

print("Feature scaling completed.")
print(df_scaled[['Amount_Scaled', 'Time_Scaled']].head())
```

```
Feature Scaling
Feature scaling completed.
   Amount_Scaled  Time_Scaled
0       2.339235    -1.995772
1      -0.746123    -1.995772
3       1.790745    -1.995750
4       0.667098    -1.995729
5      -0.725544    -1.995729
```

Preprocessing: Imbalance & Dimensionality

Class Imbalance (SMOTE)

- **Operation:** Used SMOTE to generate synthetic samples for the minority (Fraud) class.
- **Justification:** Balances data (0: 257,590 → 1: 410), preventing bias toward Non-Fraud and improving fraud detection accuracy.

```python
# 4. Handling Class Imbalance
print("Balancing Dataset using SMOTE")
X = df_scaled.drop(columns=['Class'])
y = df_scaled['Class']

smote = SMOTE(random_state=42)
X_resampled, y_resampled = smote.fit_resample(X, y)

print("SMOTE applied successfully.")
print(f"Before SMOTE: {y.value_counts().to_dict()}")
print(f"After SMOTE: {pd.Series(y_resampled).value_counts().to_dict()}")
```

```
Balancing Dataset using SMOTE
SMOTE applied successfully.
Before SMOTE: {0: 257590, 1: 410}
After SMOTE: {0: 257590, 1: 257590}
```

Dimensionality Reduction (PCA)

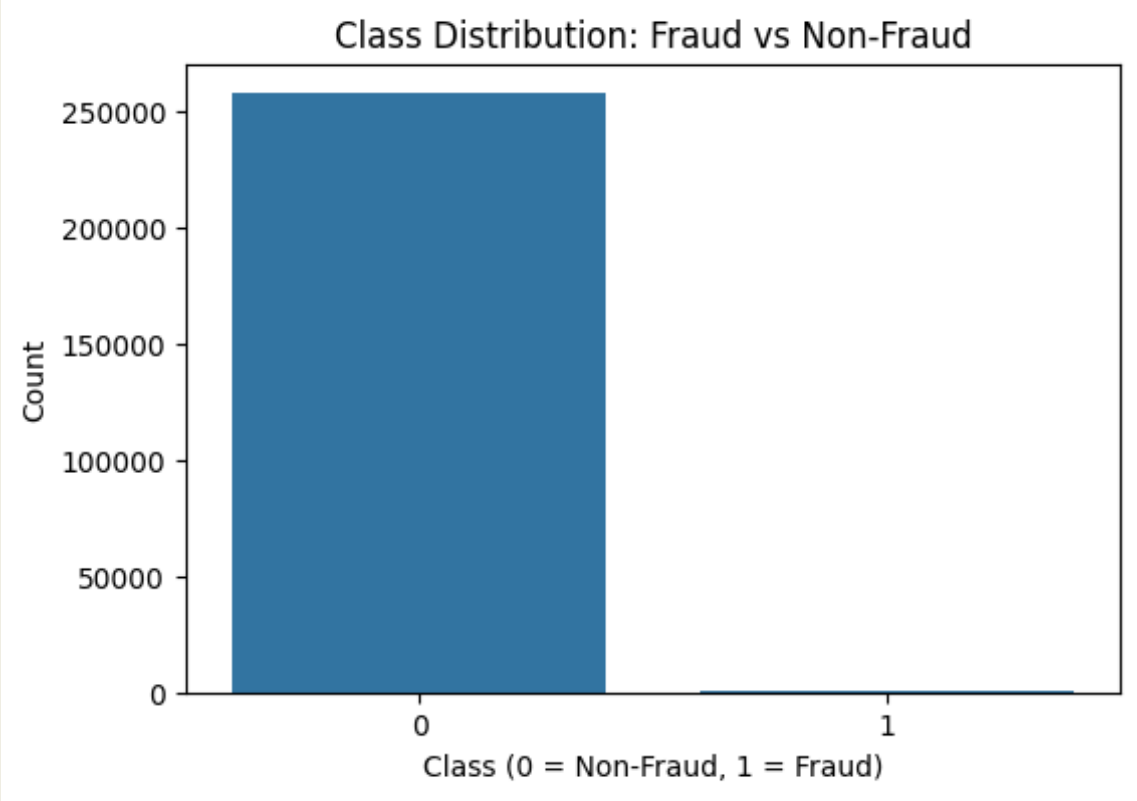
- **Operation:** Reduced 29 features to 10 principal components using PCA.
- **Justification:** Lowers complexity, minimizes multicollinearity, and speeds up training while preserving key variance.

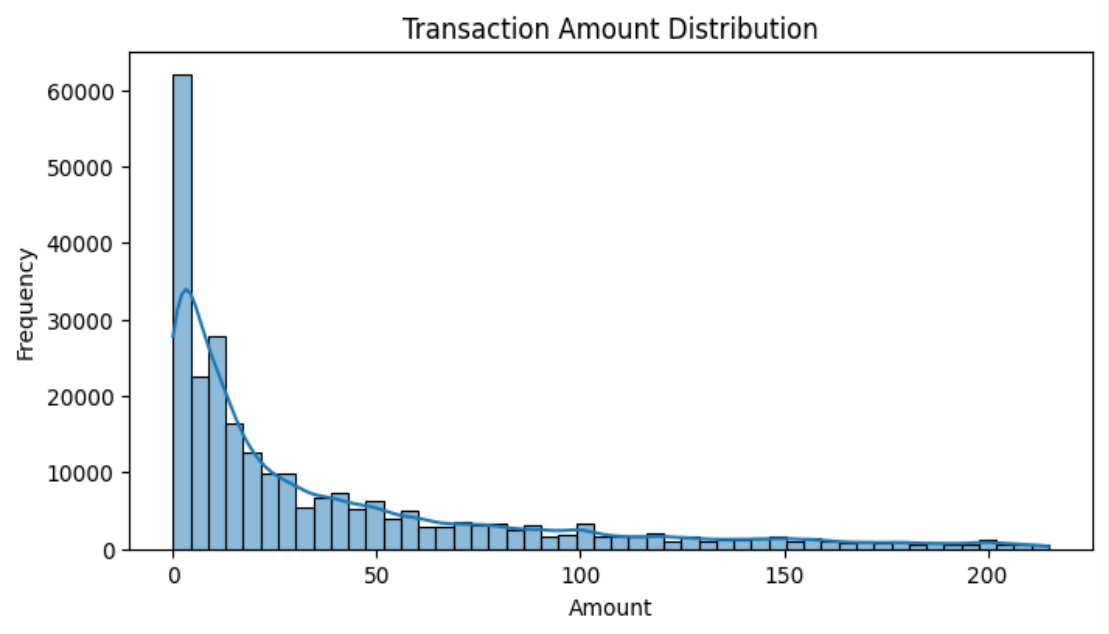
```python
# 5. Dimensionality Reduction using PCA
print("Applying PCA")
pca = PCA(n_components=10, random_state=42)
X_pca = pca.fit_transform(X_resampled)

pca_columns = [f'PC{i+1}' for i in range(X_pca.shape[1])]
X_pca_df = pd.DataFrame(X_pca, columns=pca_columns)
X_pca_df['Class'] = y_resampled.values

print("PCA completed. Transformed data shape:", X_pca_df.shape)
print(X_pca_df.head())
```

```
Applying PCA
PCA completed. Transformed data shape: (515180, 11)
         PC1       PC2       PC3       PC4       PC5       PC6       PC7
0   9.942520 -1.053027  1.195001 -1.979163 -0.006555 -1.897524  1.985533
1   9.984023 -0.862948  1.009251 -0.571955 -0.270012  0.716943  0.979063
2  10.091527 -1.156454  1.456584 -1.960673 -0.148176 -1.129500  1.111155
3   9.909246 -1.331455  1.373230 -1.798343  0.465469 -1.165637  1.704468
4   9.884361 -0.996709  1.201875 -0.767745 -0.048077 -0.689987  1.337487

        PC8       PC9      PC10  Class
0 -1.034420 -1.049338 -0.968242      0
1 -0.396851 -0.769847 -0.850949      0
2 -1.329221  0.199386 -1.062689      0
3 -1.046982 -0.118028  0.447124      0
4 -1.647769 -0.092804 -0.604674      0
```

Data Visualization

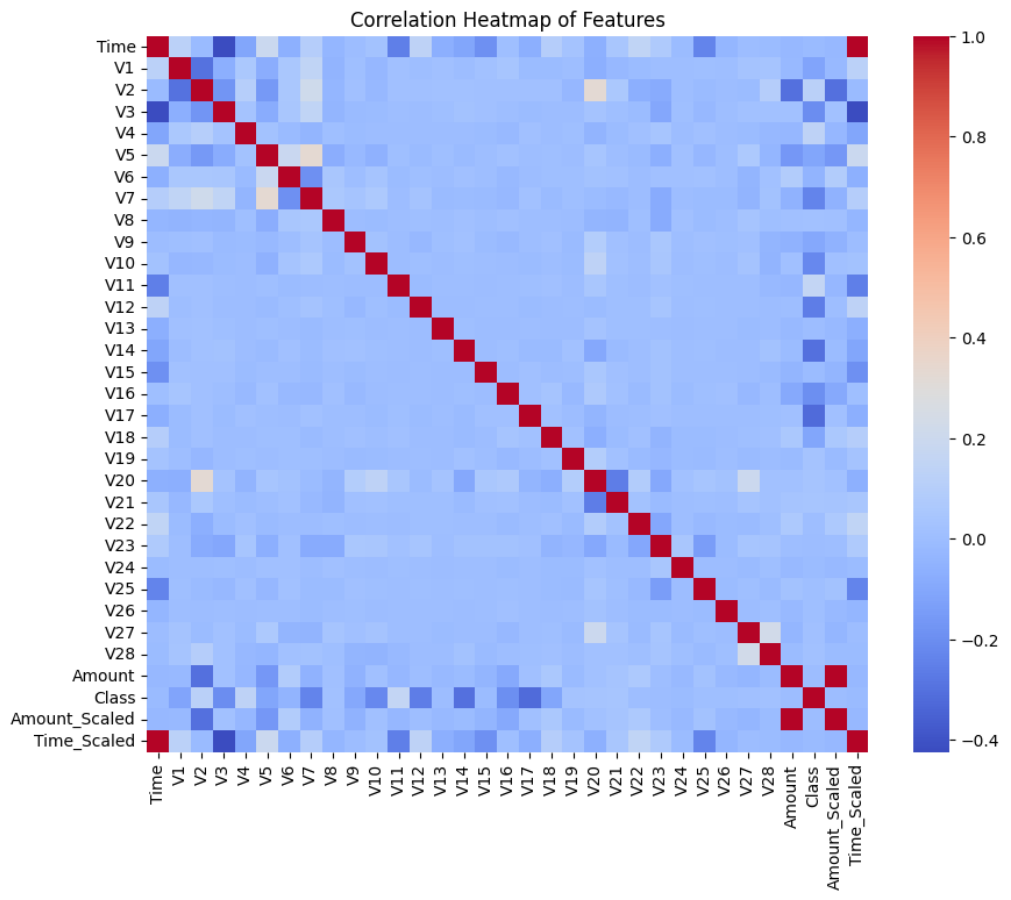
The below bar chart shows an extreme imbalance between classes — the majority (Class 0: Non-Fraud) greatly outweighs the minority (Class 1: Fraud).





The above histogram shows that most transactions are of low monetary value, while a few high-value transactions exist on the long right tail.

Data Visualization



- The heatmap illustrates correlations among PCA-transformed features (V1–V28) and scaled ‘Amount’ and ‘Time’. Strong diagonal values indicate PCA successfully reduced multicollinearity.

Preliminary Inferences & Aid to Analysis

Key Observations:

- **No Missing Data:** Confirms data integrity; allows direct feature engineering.
- **Severe Imbalance:** Fraud (Class 1) extremely rare — key challenge in modeling.
- **Outliers (6,711):** High-value anomalies may represent legitimate large transactions or potential fraud.

How It Helps Analysis:

- Ensures balanced, clean, and standardized data for unbiased model training and effective fraud detection.

Business Importance of Preprocessing

1. **Balanced Data (SMOTE):**

- Boosts **Recall**—detects more fraud, reducing financial losses.

2. **Dimensionality Reduction (PCA):**

- Cuts features from 29 → 10, lowering computational cost by ~65%, enabling real-time fraud monitoring.

3. **Feature Scaling:**

- Prevents bias from unit magnitude; ensures fair model weighting and consistent predictions.

References & Tools

Data Source:

- Credit Card Fraud Detection Dataset (TensorFlow/Kaggle)

Software:

- Jupyter Notebook, Microsoft PowerPoint

Python Libraries:

- pandas, numpy, matplotlib, seaborn
- sklearn.preprocessing.StandardScaler
- sklearn.decomposition.PCA
- imblearn.over_sampling.SMOTE